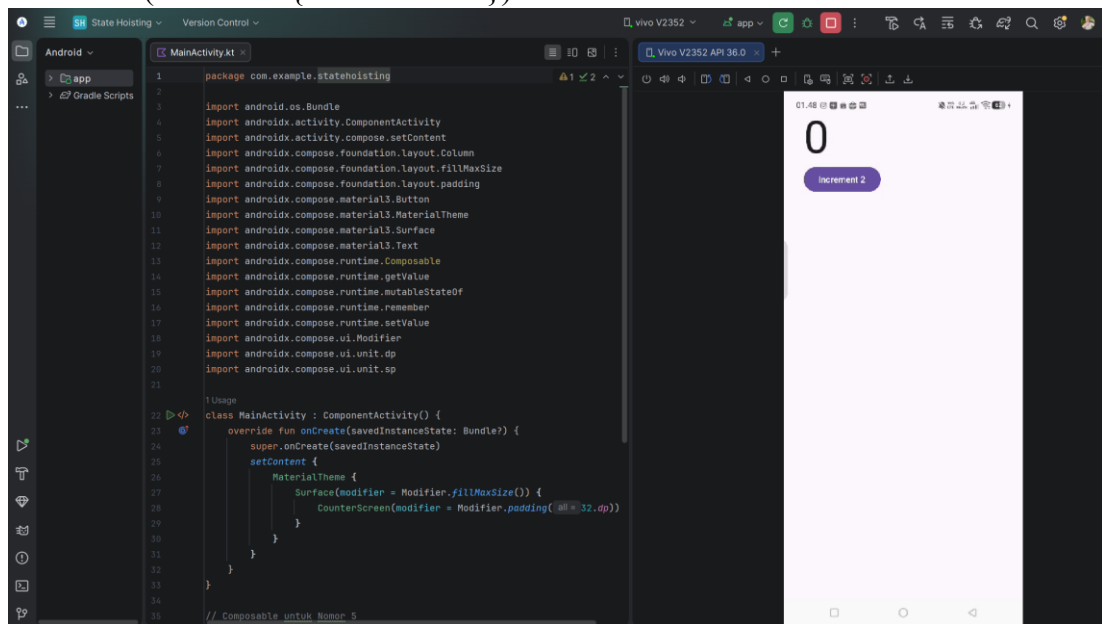


Nama: Salsabila Zalyyatul Ummah

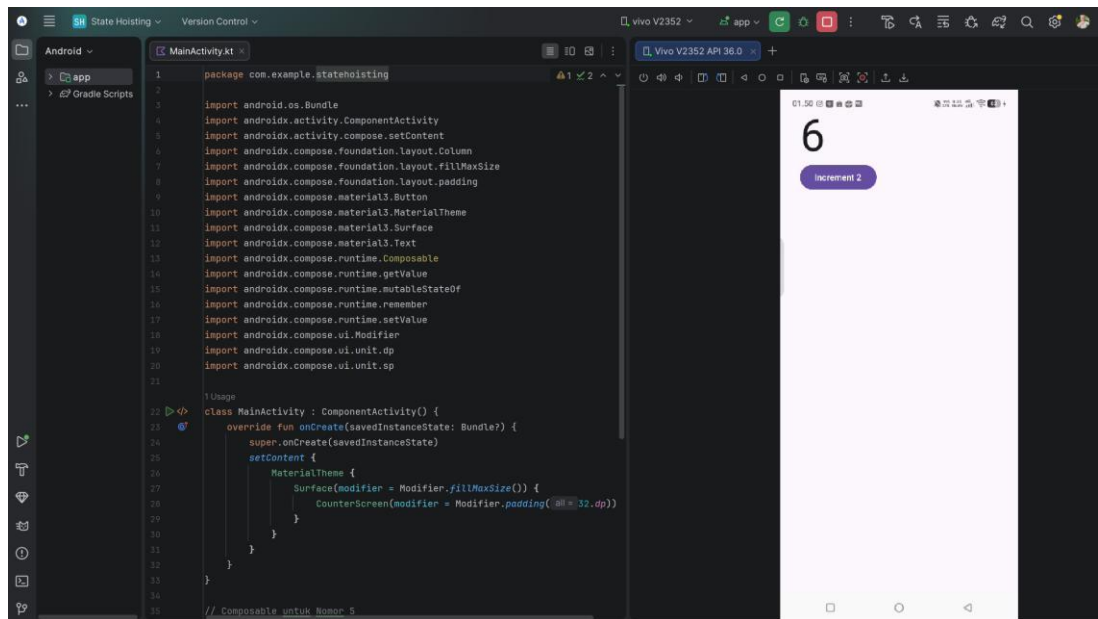
NIM: 245150607111003

PERCOBAAN 4.5.1 (State Hoisting)

1.
 - a. Angka yang ditampilkan pada layar (Text) akan bertambah 1 setiap kali tombol diklik.
 - b. Nilai variabel number diperbarui dengan sintaks `number++` saat tombol diklik. Karena number dideklarasikan menggunakan `mutableStateOf` dan `remember`, Jetpack Compose mendeteksi perubahan nilai state tersebut dan memicu proses recomposition untuk memperbarui tampilan UI Text secara otomatis.
 - c. Variabel number (`var number by remember { mutableStateOf(0) }`).
 - d. Variabel label (`val label by remember { mutableStateOf("Increment") }`).
5.
 - a. Di dalam fungsi Composable `CounterScreen`:
 1. Ubah deklarasi label awal menjadi:
`val label by remember { mutableStateOf("Increment 2") }`
 2. Ubah fungsi di dalam `Button(onClick = { ... })` menjadi:
`Button(onClick = { number += 2 })`

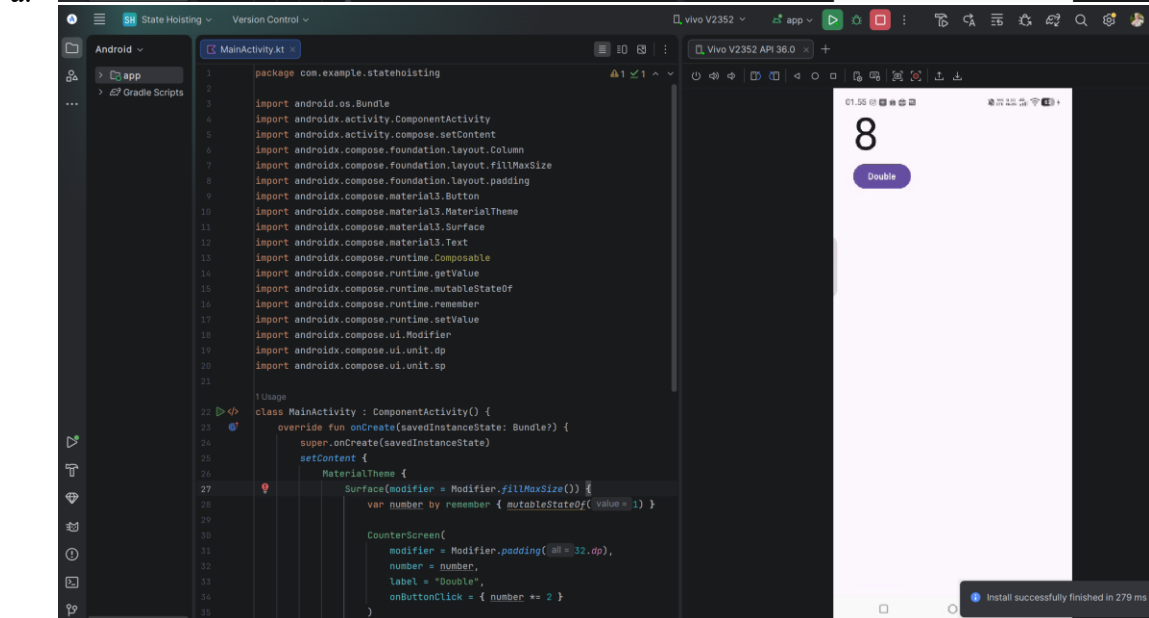
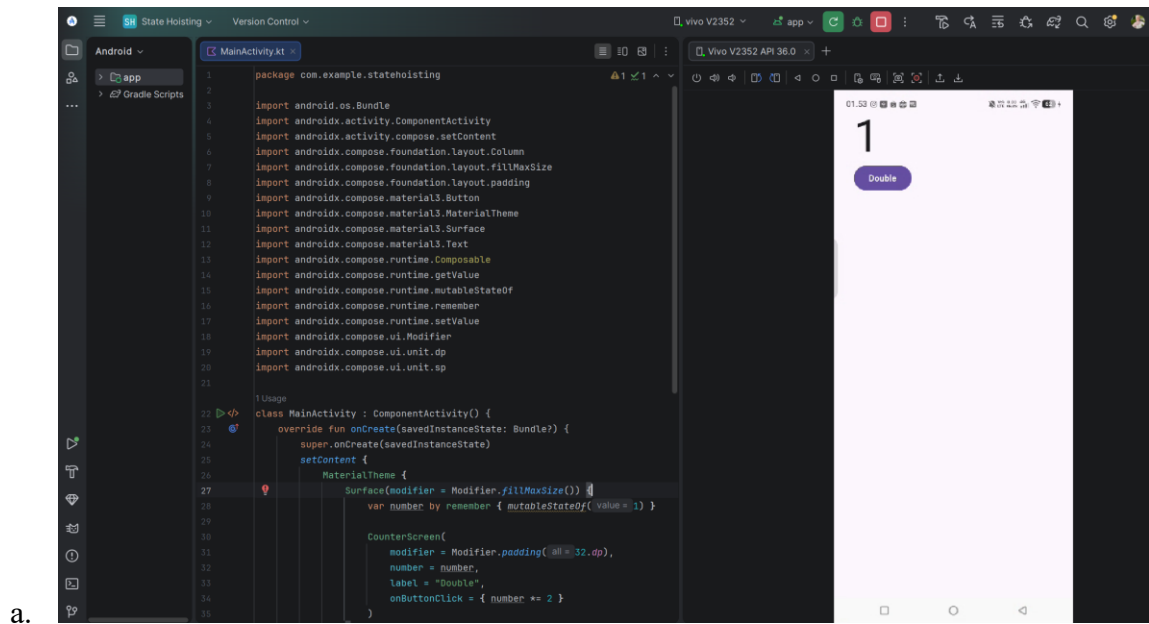


b.



- c.
 - a. Ya, program berjalan dengan baik dan menampilkan perilaku yang sama persis seperti pada langkah nomor 1.
 - b. Proses increment dilakukan pada fungsi induk/caller, yaitu di dalam pemanggilan fungsi onCreate() pada blok lambda parameter onClick = { number++ }.
 - c. Dideklarasikan di luar CounterScreen, yaitu di dalam fungsi onCreate() sebelum pemanggilan CounterScreen.
8.
 - d. Bagian kode program di dalam onCreate() setelah langkah 8(a), 8(b), dan 8(c) dilakukan:
 // Deklarasi state dengan nilai awal 1
 var number by remember { mutableStateOf(1) }

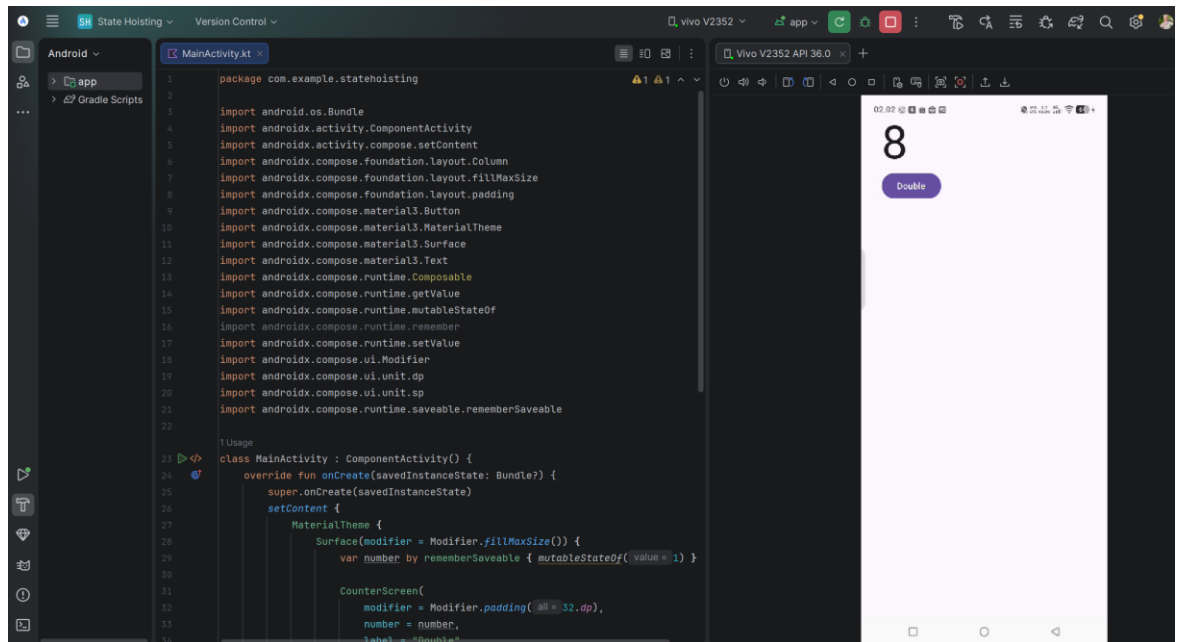
 // Pemanggilan CounterScreen dengan label "Double" dan lambda menggandakan nilai (number * 2)
 CounterScreen(
 modifier = Modifier.padding(32.dp),
 number = number,
 label = "Double",
 onClick = { number *= 2 }
)
- 9.



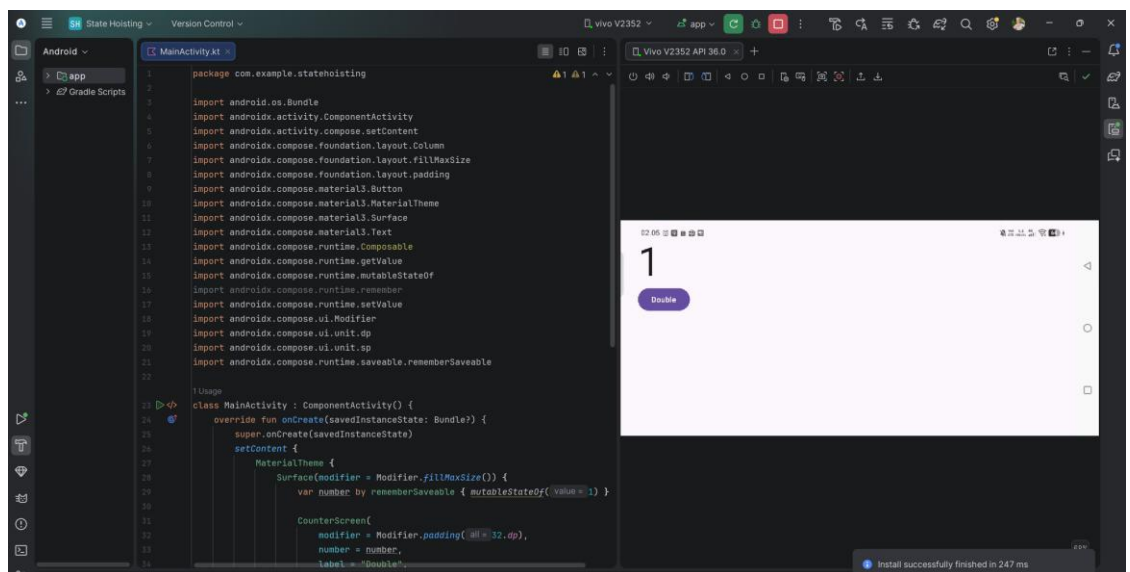
- b. Seluruh perubahan dilakukan pada level induk/caller di dalam fungsi onCreate().
 - c. Tidak ada perubahan sama sekali di dalam fungsi Composable CounterScreen. CounterScreen tetap menggunakan struktur signature dan body yang sama seperti pada langkah nomor 6.
10. Membuktikan bahwa dengan menerapkan State Hoisting, komponen UI (CounterScreen) menjadi bersifat Stateless. Manfaat utamanya adalah:
1. Reusability (Dapat Digunakan Kembali): Komposable CounterScreen menjadi fleksibel dan tidak terikat pada satu logika data saja.
 2. Separation of Concerns: Memisahkan tampilan UI dari logika bisnis. Jika ada perubahan kebutuhan antarmuka (misal: tombol dari *increment* menjadi *double*, atau nilai awal dari 0 menjadi 1), kita hanya perlu mengubah logika pada pemanggilnya (onCreate) tanpa perlu mengubah atau merusak kode internal UI CounterScreen.

PERCOBAAN 4.5.2 (Perubahan Konfigurasi)

1.



2.

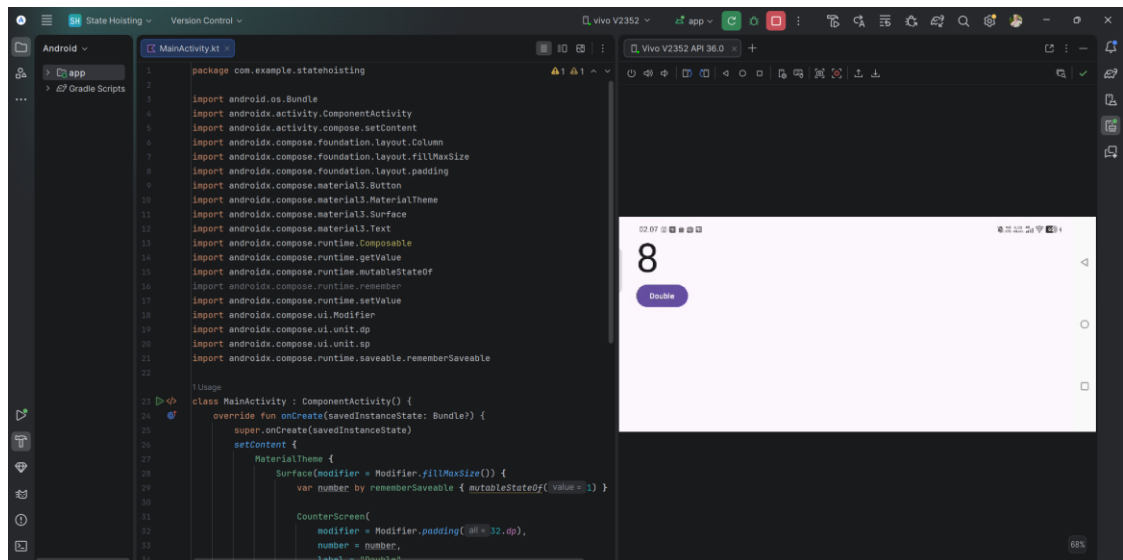


a.

b. Angka yang ditampilkan: 1 (nilai awal).

Alasan: Konstruk remember hanya berfungsi untuk menyimpan nilai state selama proses recomposition (perubahan UI biasa). Ketika terjadi Configuration Change (seperti rotasi layar dari portrait ke landscape), sistem Android akan menghancurkan (*destroy*) dan membuat ulang (*recreate*) Activity. Karena Activity dibuat ulang dari awal, nilai variabel yang disimpan dengan remember akan terhapus dari memori dan diinisialisasi kembali ke nilai awalnya (1).

3.



- a.
- b. Angka yang ditampilkan: 8 (atau nilai angka terakhir sebelum layar diputar).
Alasan: Konstruk `rememberSaveable` secara otomatis menyimpan nilai state ke dalam mekanis Bundle / `SavedInstanceState` milik sistem Android saat terjadi pembuatan ulang Activity akibat rotasi layar. Ketika Activity dibuat kembali di posisi landscape, `rememberSaveable` memulihkan (*restore*) nilai state terakhir tersebut dari Bundle.
- c. `remember`: Hanya menjaga state tetap bertahan saat terjadi *recomposition* UI biasa. State akan hilang jika terjadi *Configuration Change* (rotasi layar, perubahan tema gelap/terang, pergantian bahasa).
`rememberSaveable`: Menjaga state tetap bertahan baik saat *recomposition* maupun saat terjadi *Configuration Change* (serta pemulihan proses sementara oleh sistem) dengan menyimpan nilainya ke dalam Bundle.
4. Contoh Kasus Nyata:
 1. Form Input Teks (Pendaftaran / Login / Pencarian): Mengisikan teks nama, email, atau kueri pencarian pada komponen `TextField`.
 2. Formulir Isian Banyak Pilihan: Status centang (*checkbox*), pilihan *radio button*, atau *toggle switch*.
 3. Kuis Online / Survey: Pilihan jawaban sementara yang sudah dipilih pengguna sebelum dikirim (*submit*).

Manfaat yang Diperoleh:

- Meningkatkan User Experience (UX): Mencegah pengguna merasa frustrasi karena harus mengetik atau memilih ulang data dari awal hanya karena ponselnya tidak sengaja berotasi atau berpindah mode tampilan.
- Mencegah Kehilangan Data Ringan: Menjaga integritas data sementara di tingkat UI agar aplikasi terasa lebih responsif dan andal.

PERCOBAAN 4.5.3 (LaunchedEffect)

1.
 - a. Layar menampilkan angka 1 di bagian atas, tombol bertuliskan "Double", dan di bagian bawah layar muncul sebuah Snackbar bertuliskan "Number 1 is shown!".

- b. Karena `LaunchedEffect` dipicu dan dieksekusi secara otomatis saat `Composable` pertama kali masuk ke dalam komposisi (UI tree).
- c. Pemicunya adalah eksekusi blok `LaunchedEffect(number)` saat inisialisasi awal UI. Kode program yang memicu adalah:

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}
```

2.

- a. Angka pada layar berubah menjadi 2, dan muncul pesan baru di Snackbar bertuliskan "Number 2 is shown!".
- b. Ketika tombol "Double" diklik, nilai variabel `number` berubah dari 1 menjadi 2. Karena `number` dipasang sebagai parameter *key* pada `LaunchedEffect(number)`, setiap kali nilai *key* tersebut berubah, `LaunchedEffect` akan membatalkan coroutine yang sedang berjalan dan memanggil kembali blok kodenya dengan nilai `number` yang baru (2).

1.

- a. Layar awalnya menampilkan angka 1 dan Snackbar awal "Number 1 is shown!".
- b. Setelah 2 detik, angka pada layar otomatis berubah secara acak menjadi angka antara 100 sampai 1000 (misalnya 542). Snackbar juga otomatis diperbarui menampilkan pesan "Number 542 is shown!".
- c. Pemicunya adalah pemanggilan fungsi asinkron `Repo.getData()` di dalam `LaunchedEffect(Unit)`. Fungsi ini menunda eksekusi selama 2 detik (`delay(2000)`), mengambil angka acak, lalu memperbarui variabel state `number`. Kode program yang memicu adalah:

```
LaunchedEffect(Unit) {
    number = Repo.getData()
}
```

Ketika variabel `number` diperbarui oleh `Repo.getData()`, perubahan state tersebut memicu `LaunchedEffect(number)` pada percobaan pertama untuk menampilkan Snackbar baru.

2.

- a. `LaunchedEffect(Unit)`: Menggunakan *key* konstan `Unit` yang nilainya tidak pernah berubah. Digunakan untuk tugas yang hanya perlu dijalankan satu kali saja selama siklus hidup `Composable`.
`LaunchedEffect(state)`: Menggunakan variabel state sebagai *key*. Blok kode di dalamnya akan dijalankan ulang setiap kali nilai variabel state tersebut mengalami perubahan.
- b. `LaunchedEffect(Unit)`: Dijalankan tepat satu kali saja, yaitu pada saat `Composable` pertama kali masuk ke dalam komposisi UI (initial composition). Cocok digunakan untuk mengambil data dari API/database saat layar pertama kali dibuka.
`LaunchedEffect(state)`: Dijalankan saat pertama kali masuk ke dalam komposisi UI DAN dieksekusi ulang setiap kali parameter state yang menjadi *key*-nya berubah nilai. Cocok digunakan untuk memicu efek samping (seperti menampilkan Toast/Snackbar, animasi, atau navigasi) berdasarkan perubahan data tertentu.